

FinInsight RAG (Financial & SQL Analytics Hybrid)

Explanation of this project in leman language

What Does FinInsight RAG Do in Plain English?

Imagine a financial analyst working at a company. They constantly deal with two very different types of information:

1. **Numbers in Databases (Structured Data):** E.g., "What were the exact revenue numbers for Q1, Q2, and Q3?"
2. **Text in PDFs/Documents (Unstructured Data):** E.g., "Why did the CEO say profits dropped in Q3?" or "What risk factors were mentioned regarding supply chains?"

Normally, an AI tool can only handle **one** of these at a time.

FinInsight RAG acts like a smart financial assistant that handles both seamlessly:

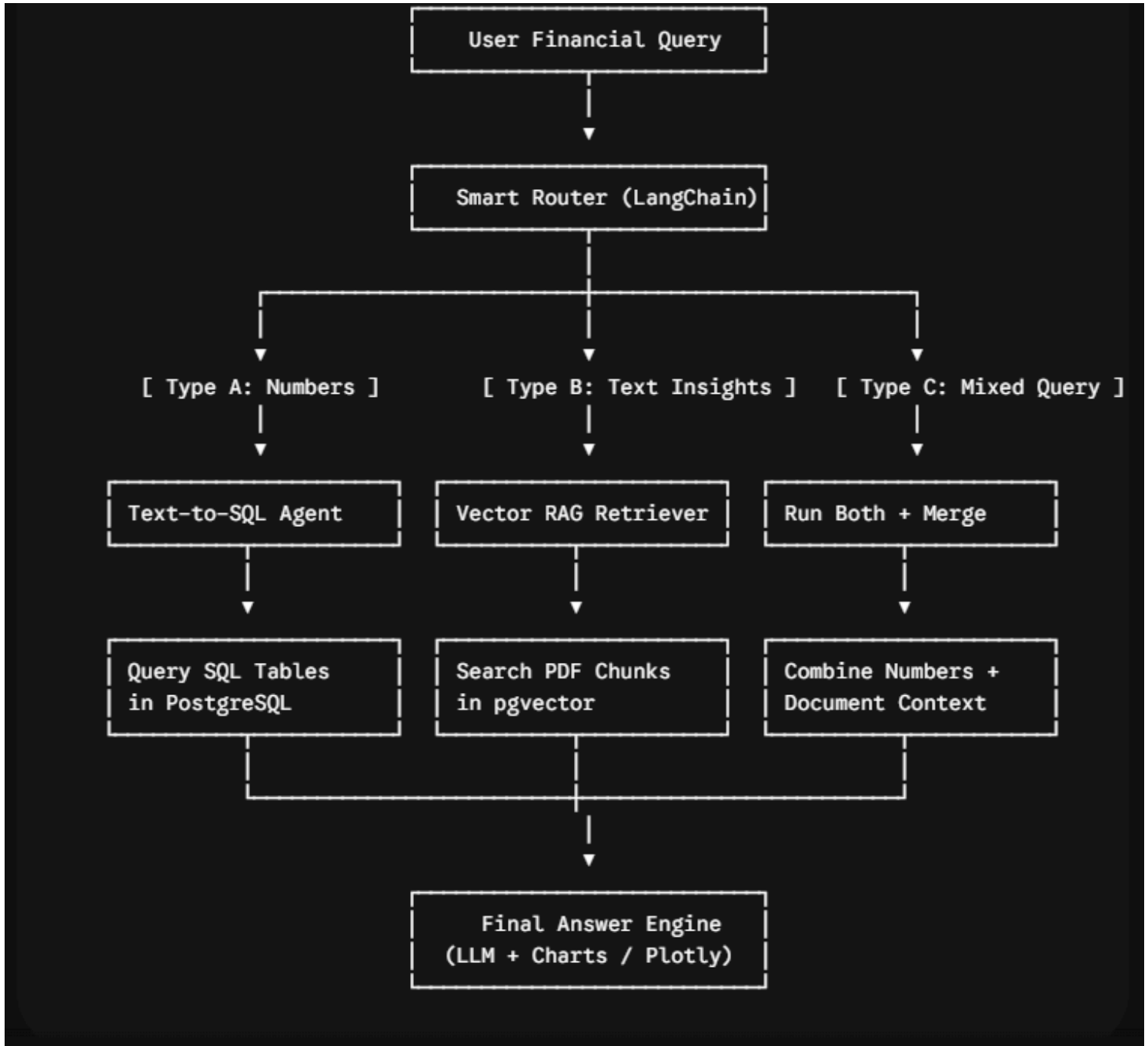
- When you ask a question about **numbers**, it writes a SQL query to extract exact figures from a database.
- When you ask a question about **explanations or strategy**, it uses Vector Search (RAG) to find matching paragraphs in PDFs.
- When you ask a **complex mixed question** (e.g., "Show me Q3 revenue and explain why it changed compared to Q2"), it automatically combines data from **both sources** into one clear answer with charts!

Tech Stack for this Project

Component	Technology Choice	Role
Framework	LangChain	Manages prompt templates, SQL Tool agents, and Vector RAG chains.
Database & Vector Store	PostgreSQL + pgvector	Holds both SQL financial tables and text vector embeddings in one place.

LLM Engine	OpenAI (gpt-4o / gpt-4o-mini)	Generates SQL queries, reads document context, and synthesizes answers.
PDF Parser	Unstructured.io or PyPDF	Extracts text and tables from financial PDFs (e.g., SEC 10-K filings, earnings transcripts).
Backend API	FastAPI	Exposes REST endpoints to query the system and upload files.
Frontend UI	Streamlit or Next.js	Renders dynamic SQL charts/plots alongside text answers.

How this Project Works



Execution Plan for this Project

Phrase 1 : Database Setup

Phrase 2 : Ingestion Pipeline

Phrase 3 : RAG & SQL Engines

Phrase 4 : API & Frontend

Step 1: Database & Environment Setup

Set up a unified **PostgreSQL** database using the pgvector extension. This allows you to host relational tables (for numbers) and vector tables (for PDF embeddings) in the same database.

1. Structured Tables (**financial_metrics**):

- Columns: id, company, year, quarter, revenue, operating_expense, net_profit, margin.

2. Vector Table (**document_embeddings**):

- Columns: id, document_name, chunk_text, embedding (vector format), page_number.

Step 2: Data Ingestion Pipeline (PDF Parsing & Chunking)

Build a module using Python to parse and upload financial data:

1. CSV / Structured Data: Loads company financial metrics straight into the PostgreSQL relational tables.

2. PDF Processing (Unstructured Data):

- Uses **PyPDF** or **Unstructured.io** to extract text from 10-K filings, annual reports, and transcripts.
- Uses LangChain's RecursiveCharacterTextSplitter (chunk size ~1000 characters with 150 overlap).
- Generates vector embeddings using OpenAI (text-embedding-3-small) or HuggingFace (bge-small-en).
- Stores chunks and vector representations into pgvector.

Step 3: Building the Intelligence Core (LangChain)

A. Text-to-SQL Agent

- Uses LangChain's SQL Toolkit (SQLDatabaseChain).
- Reads the database schema (table column names, data types).
- When asked a numerical question, the LLM generates a SQL query, runs it securely, and formats the table results.

B. Vector RAG Retriever

- Connects to pgvector vector store.
- Uses **Similarity Search** or **MMR (Maximal Marginal Relevance)** to retrieve the top 3-5 relevant document paragraphs.
- Feeds context to the LLM to write factual, citation-backed text summaries.

C. Smart Router & Orchestrator

- Uses LangChain's RouterRunnable or conditional chains.
- Inspects the incoming prompt using an LLM classifier and directs the request to:
 - SQL_CHAIN
 - VECTOR_CHAIN
 - HYBRID_CHAIN (runs both in parallel and merges outputs)

Step 4: Backend API Layer (FastAPI)

Expose the backend functionality via RESTful endpoints:

- **POST /upload_pdf**: Accepts financial PDF files, chunks them, and stores embeddings in pgvector.
- **POST /upload_csv**: Ingests structured spreadsheets into PostgreSQL relational tables.
- **POST /query**: Accepts natural language questions, runs the Router, and returns JSON containing:
 - Final textual answer
 - SQL query generated (if applicable)
 - Raw data table / chart data points
 - Source document citations (page numbers, document names)

Step 5: Interactive Frontend UI (Streamlit or Next.js)

Build a clean interface for interaction:

- **Left Sidebar**: Upload PDFs/CSVs and manage vector stores.
- **Main Chat Interface**:
 - Displays user prompt and response.
 - Shows auto-generated charts (using **Plotly**) whenever numerical data is returned from SQL.
 - Collapsible source drawer showing exact PDF text snippets and page references.